

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CH. Durga Prasad

1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(30)
);
```

```
CREATE TABLE Faculty (
    FacultyID INT PRIMARY KEY,
    FacultyName VARCHAR(30),
    Phone VARCHAR(15)
);
```

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(30),
    FacultyID INT,
    FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID)
);
```

```
CREATE TABLE Enrollment (
    StudentID INT,
```

```

    CourseID INT,
    PRIMARY KEY(StudentID, CourseID),
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY(CourseID) REFERENCES Course(CourseID)
);
INSERT INTO Student VALUES
(101,'Rahul'),
(102,'Priya');

INSERT INTO Faculty VALUES
(1,'Ramesh','9876543210'),
(2,'Suresh','9876501234');

INSERT INTO Course VALUES
(201,'DBMS',1),
(202,'Python',2);

INSERT INTO Enrollment VALUES
(101,201),
(102,202);
SELECT S.StudentName,C.CourseName,F.FacultyName
FROM Enrollment E
JOIN Student S ON E.StudentID=S.StudentID
JOIN Course C ON E.CourseID=C.CourseID
JOIN Faculty F ON C.FacultyID=F.FacultyID;

```

Output:

StudentName	CourseName	FacultyName
Rahul	DBMS	Ramesh
Priya	Python	Suresh

2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach

```

CREATE TABLE Employee(
    EmployeeID INT,
    EmployeeName VARCHAR(30),
    DepartmentID INT
);
INSERT INTO Employee VALUES
(101,'Rahul',1),

```

```
(101,'Rahul',2),
(102,'Priya',1),
(103,'Kiran',3),
(103,'Kiran',4);
SELECT EmployeeID,EmployeeName
FROM Employee
GROUP BY EmployeeID,EmployeeName
HAVING COUNT(DepartmentID)>1;
```

Output:

EmployeeID	EmployeeName
101	Rahul
103	Kiran

3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

```
CREATE TABLE StudentCourse(
    StudentID INT,
    CourseID INT,
    Faculty VARCHAR(30)
);
INSERT INTO StudentCourse VALUES
(101,201,'Ramesh'),
(102,202,'Suresh'),
(103,201,'Ramesh');
SELECT * FROM StudentCourse;
```

Output:

StudentID	CourseID	Faculty
101	201	Ramesh
102	202	Suresh
103	201	Ramesh

4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

```
CREATE TABLE Account(
    AccountNo INT PRIMARY KEY,
    CustomerName VARCHAR(30),
```

```

        Balance DECIMAL(10,2)
    );
INSERT INTO Account VALUES
(1001,'Rahul',25000),
(1002,'Priya',40000);
CREATE VIEW CustomerView AS
SELECT AccountNo, CustomerName, Balance
FROM Account;
SELECT * FROM CustomerView;

```

Output:

AccountNo	CustomerName	Balance
1001	Rahul	25000.00
1002	Priya	40000.00

5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

```

CREATE TABLE Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(30)
);

```

```

CREATE TABLE Supplier(
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(30)
);

```

```

CREATE TABLE Product(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(30),
    SupplierID INT,
    FOREIGN KEY(SupplierID) REFERENCES Supplier(SupplierID)
);

```

```

CREATE TABLE Orders(
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    ProductID INT,
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY(ProductID) REFERENCES Product(ProductID)
);

```

```
);
INSERT INTO Customer VALUES
(1,'Rahul'),
(2,'Priya');
```

```
INSERT INTO Supplier VALUES
(11,'ABC Traders');
```

```
INSERT INTO Product VALUES
(101,'Laptop',11),
(102,'Mouse',11);
```

```
INSERT INTO Orders VALUES
(1001,1,101),
(1002,2,102);
SELECT C.CustomerName,P.ProductName,S.SupplierName
FROM Orders O
JOIN Customer C ON O.CustomerID=C.CustomerID
JOIN Product P ON O.ProductID=P.ProductID
JOIN Supplier S ON P.SupplierID=S.SupplierID;
```

Output:

CustomerName	ProductName	SupplierName
Rahul	Laptop	ABC Traders
Priya	Mouse	ABC Traders

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(30)
);
```

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)
);
```

```
INSERT INTO Customer VALUES
(1,'Rahul'),
(2,'Priya'),
(3,'Kiran');
```

```

INSERT INTO Orders VALUES
(101,1),
(102,1),
(103,1),
(104,2),
(105,2),
(106,1);
SELECT C.CustomerName, O.OrderID
FROM Customer C
JOIN Orders O
ON C.CustomerID = O.CustomerID;

```

```

SELECT CustomerID, COUNT(OrderID) AS TotalOrders
FROM Orders
GROUP BY CustomerID
HAVING COUNT(OrderID) > 3;

```

Output:

CustomerName	OrderID
Rahul	101
Rahul	102
Rahul	103
Rahul	106
Priya	104
Priya	105

CustomerID	TotalOrders
1	4

7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

```

CREATE TABLE Student(
    StudentID INT,
    Hobby VARCHAR(30),
    Language VARCHAR(30)
);
INSERT INTO Student VALUES
(101,'Cricket','English'),
(101,'Cricket','Hindi'),

```

```
(101,'Music','English'),
(101,'Music','Hindi');
SELECT * FROM Student;
```

Output:

StudentID	Hobby	Language
101	Cricket	English
101	Cricket	Hindi
101	Music	English
101	Music	Hindi

8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

```
CREATE TABLE Employee(
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(30) NOT NULL,
    Salary DECIMAL(10,2) CHECK(Salary > 0),
    Email VARCHAR(40) UNIQUE
);
INSERT INTO Employee VALUES
(101,'Rahul',35000,'rahul@gmail.com'),
(102,'Priya',42000,'priya@gmail.com');
SELECT * FROM Employee;
```

Output:

EmployeeID	EmployeeName	Salary	Email
101	Rahul	35000.00	rahul@gmail.com
102	Priya	42000.00	priya@gmail.com

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

```
CREATE TABLE SupplierProjectPart(
    Supplier VARCHAR(30),
    Project VARCHAR(30),
    Part VARCHAR(30)
);
INSERT INTO SupplierProjectPart VALUES
```



```

('ABC','P1','Bolt'),
('ABC','P2','Nut'),
('XYZ','P1','Bolt');
SELECT * FROM SupplierProjectPart;

```

Output:

Supplier	Project	Part
ABC	P1	Bolt
ABC	P2	Nut
XYZ	P1	Bolt

10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

```

CREATE TABLE Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(30)
);

```

```

CREATE TABLE Orders(
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)
);

```

```

INSERT INTO Customer VALUES
(1,'Rahul'),
(2,'Priya');

```

```

INSERT INTO Orders VALUES
(101,1),
(102,2),
(103,1);

```

```

SELECT O.*
FROM Orders O
JOIN Customer C
ON O.CustomerID = C.CustomerID;
CREATE INDEX idx_customer
ON Orders(CustomerID);

```

Output:

+-----+-----+	
OrderID	CustomerID
+-----+-----+	
101	1
103	1
102	2
+-----+-----+	